

Programming Assignment 2

Filename: pa2.c

You must include the following commented lines at the beginning of your code to declare authorship. Submissions that do not include these lines will receive a 10% deduction.

```
/* COP 3502C PA2  
This program is written by: Your Full Name */
```

In this activity, you will practice designing and implementing recursive algorithms in C, focusing on the following:

- Writing recursive functions that perform counting and checking tasks
- Implementing permutation generation using the `is_used` array technique
- Working with structures, pointers, and pointer reassignment in recursive contexts

Background

The annual *Grand Meow-Sters Cat Team Challenge* is fast approaching. You are responsible for assembling the strongest possible teams of cats. You will organize $n \times c$ cats into n teams, each consisting of c cats. Each cat is evaluated across five traits: Agility, Beauty, Confidence, Determination, and Elegance (ABCDE). The order of cats within a team matters, as each position applies a different scoring multiplier.

In addition to individual traits, team performance is influenced by bonuses, penalties, and special interactions between cats. Some cats work particularly well together, while others do not. A score is computed for each team, and the team scores are then combined to produce an overall score (i.e., permutation score).

You will read information about a set of cats and assign them to teams of the required size. Each team must contain exactly the specified number of cats. Your program must systematically evaluate all possible permutations of the cats, compute the corresponding overall score for each arrangement, and identify the arrangement that yields the maximum combined team score.

The final output represents the optimal grouping and ordering of cats.

Variables and Context

Each cat is represented by a `Cat` structure that contains dynamically allocated fields for the cat's name and breed, a fixed-size array of five trait scores, and a base score computed as the sum of those five traits.

Some cats are designated as rivals. Rival relationships are stored using a `Rivals` structure, which holds pointers to the two cats involved. A rivalry penalty is applied if, within a team, a pair of rival cats appears in adjacent positions. This penalty is incurred at most once per team, regardless of how many adjacent rival pairs occur. Rivalries are symmetric.

Your program must generate permutations using the `is_used` array permutation technique covered in class. A single one-dimensional integer array represents a permutation of cats, where contiguous segments of the array correspond to individual teams.

For example, when 9 cats are divided into 3 teams of 3 cats each, a permutation array of size 9 is interpreted as follows:

- Team 0: indices 0–2
- Team 1: indices 3–5
- Team 2: indices 6–8

You should not permute the `Cat` structures themselves. Instead, the permutation array stores indices used to access cat information from the global `cats` array.

A global `POSITION_BONUS` array provides percentage-based bonuses for cat positions within a team. Only the first c values of this array are used, where c is the team size.

You are required to use exactly four global variables and no others:

```
Cat *cats;
Rivals *rivals;
double bestPermScore;
int **tracker;
```

The `tracker` array stores the best arrangement found so far. It is organized as a two-dimensional array where rows represent teams and columns represent cat positions within a team. When a permutation with a higher overall score is discovered, you must copy values from the one-dimensional permutation array into this two-dimensional structure.

Consider 9 cats divided into 3 teams of 3 cats each. Suppose the permutation array is:

```
perm = { 6, 4, 1, 7, 0, 3, 8, 5, 2 }
```

After updating the `tracker` array (of size 3×3), it will contain:

```
tracker = {
    { 6, 4, 1 }, // Team 0
    { 7, 0, 3 }, // Team 1
    { 8, 5, 2 }  // Team 2
}
```

Each row of `tracker` corresponds to a team, and each column corresponds to a position within that team. For example, `tracker[0]` stores the cats assigned to Team 0, while `tracker[1]` stores the cats assigned to Team 1.

Required Data Structures

You must use the following structures in this assignment. You are allowed to declare more structures if needed. However, you are not allowed to change the required structures.

```
typedef struct Cat_s {
    char *name;           // Cat's name (dynamic string)
    char *breed;          // Cat's breed (dynamic string)
    int scores[MAX_SCORES]; // Fixed-sized array for the cat's 5
                           // traits scores
    int baseScore;        // Sum of the cat's 5 traits scores
} Cat;

typedef struct Rivals_s {
    Cat *cat1;            // Pointer to cat1
    Cat *cat2;            // Pointer to cat2
} Rivals;
```

Required Recursive Functions

In addition to the required recursive used-array permutation function, you are required to implement three recursive functions to use during the team scoring process. These functions are as follows:

```
// Counts the number of high-performing traits among all cats in a team
countHighPerformerTraits(...)

// Determines whether a team qualifies for a synergy bonus
doesSynergyBonusApply(...)

// Determines whether a team incurs a rivalry penalty
doesRivalPenaltyApply(...)
```

You are responsible for designing the function's parameters and return type, as well as defining its base cases and recursive logic.

Global Constant Variable

The following must be declared in your code:

```
#define MAX_SCORES 5
#define MAX_LEN 26

const double POSITION_BONUS[10] = {
    3.0, 5.0, 4.0, 6.0, 7.0, 2.0, 8.0, 1.0, 9.0, 1.5
};
```

Starter Code

Your task is to complete the given C program. You can access the starter code [here](#).

Team Scoring System

To compute the score for a given team, follow these three steps: first, calculate the base score; then add any applicable bonuses; and finally, subtract any applicable penalties.

Step 1: Calculate Base Score

The base score of a team is defined as the average of the base scores of all cats on that team. You must preserve full decimal precision during computation.

$$\text{Team Base Score} = \frac{1}{m} \sum_{i=1}^m B_i$$

where B_i denotes the base score of the i -th cat on the team, and m is the number of cats in that team.

Step 2: Calculate Bonus Points

High Performer Bonus

For each cat on a team, award +5 points for every trait (out of the five traits) with a score of 90 or higher.

Synergy Bonus

If every cat on the team has at least one trait score of 85 or higher, award a one-time bonus of +30 points to the team score.

Breed Diversity Bonus

If all cats on a team are of different breeds, award +10 points to the team score.

Position Bonus

For each cat on a team, add a position bonus equal to a percentage of that cat's base score. The percentage is determined by the cat's position within the team using the `POSITION_BONUS` array. Only the first c entries are used, where c is the team size.

Example: Position Bonus

The position bonus depends only on a cat's position within its team and applies regardless of which team the cat is assigned to. If a cat with a base score of 80 is placed at position 0, the position bonus added to the team score is

$$80 \times 0.03 = 2.4,$$

since `POSITION_BONUS[0] = 3.0`.

Step 3: Calculate Penalty Points

Rival Penalty

A rivalry penalty of 25 points is applied if the team's ordering contains at least one pair of rival cats in adjacent positions. This penalty is applied at most once per team, regardless of how many adjacent rival pairs occur.

- If the team size is 3 and a rival pair appears at positions 0 and 2, the cats are not adjacent and no penalty is applied.
- If the team size is 2 and the two cats are rivals, the penalty is unavoidable because the cats are necessarily adjacent.
- More generally, for any team size, a rivalry penalty is applied if and only if at least one rival pair appears in adjacent positions.

Duplicate Breed Penalty

If a team contains two or more cats of the same breed, subtract 15 points from the team score. This penalty is applied at most once per team.

Hint: This condition is the logical complement of the breed diversity bonus. Each team will receive either the diversity bonus or the duplicate breed penalty, but not both. Therefore, you only need to perform a single check to determine which outcome applies.

Permutation Score

The permutation score is defined as the overall score of a given arrangement of cats and is computed as the sum of the scores of all teams in that arrangement. The objective is to evaluate all possible permutations and select the one that yields the maximum permutation score.

Important Note

Because permutation scores are computed using floating-point arithmetic, comparisons must account for small numerical errors. A permutation replaces the current best only if its score exceeds the best score by more than an epsilon tolerance of $1e-6$. If two permutation scores differ by at most this tolerance, they are treated as equal, and the first permutation encountered is retained.

Input

The program reads input from a file named `scores.txt`. The first line contains two integers, n ($1 \leq n \leq 6$) and c ($1 \leq c \leq 10$), where n is the number of teams and c is the number of cats per team.

The next $n \times c$ lines each describe a single cat using the following format:

```
<cat_name> <cat_breed> <s0> <s1> <s2> <s3> <s4>
```

Here, `cat_name` and `cat_breed` are strings (maximum length 25) representing the cat's name and breed, respectively, and must be dynamically allocated. The values `s0` through `s4` represent the five trait scores for the cat, whose sum is used to compute the cat's base score. Cat names are guaranteed to be unique.

Following the cat information, a single integer r ($0 \leq r \leq n \times c$) specifies the number of rival pairs. The next r lines each contain two strings in the following format:

```
<cat1_name> <cat2_name>
```

Each line indicates a pair of cats that are rivals. Both names are guaranteed to refer to cats listed earlier in the input.

Important Note

You may assume that all input strings contain no whitespace.

Output

The program must output the overall score of the best permutation arrangement (formatted to two decimal places), followed by a breakdown of each team and its corresponding score (also to two decimal places). The overall score is computed as the sum of the individual team scores.

Because team order affects bonuses and penalties, cats within each team must be printed in the exact order that maximizes the overall arrangement score. This order corresponds directly to the order stored in the `tracker` array.

Finally, the output must indicate which team is the best candidate among all teams. If multiple teams have the same score, select the team that appears first. Team numbers are displayed using 1-based indexing to align with standard human numbering conventions.

The output must follow the format shown below:

```
Best Teams Grouping Score: <permutation_score>
Team 1: <cat_1> <cat_2> ... <cat_c> <team_score>
Team 2: <cat_1> <cat_2> ... <cat_c> <team_score>
...
Team <n>: <cat_1> <cat_2> ... <cat_c> <team_score>
Best Candidate: <cat_1> <cat_2> ... <cat_c>
```

Sample Input score.txt

```
3 3
Coco Ragdoll 85 90 75 80 88
Polter Ragdoll 70 85 92 78 81
Hana DSH 95 72 80 91 77
Leo Tux 88 12 71 83 79
Fudge DSH 76 81 89 92 84
Pieces DSH 82 77 86 79 93
Caesar Siamese 47 82 15 63 91
Percy DSH 28 54 76 39 12
Pikachu DSH 58 23 87 41 69
1
Coco Polter
```

Sample Output

```
Best Teams Grouping Score: 1297.89
Team 1: Caesar Fudge Polter 476.61
Team 2: Percy Coco Leo 375.49
Team 3: Pikachu Pieces Hana 445.79
Best Candidate: Caesar Fudge Polter
```

Additional Implementation Requirements

- You must use the permutation used-array technique in your solution. Otherwise, you will receive a score of 0.
- Your permutation algorithm must be recursive and must follow the approach shown in class, or a penalty will be applied.
- Your code must include the three required recursive functions.
- Your solution may not include any global variables other than the four given ones.
- For strings, you must first read the input into a static char array, then allocate the exact amount of memory needed based on its length and copy the string using strcpy function as demonstrated in class.
- You must free all the memory to receive full credit.
- Your code must have no memory leaks at the end of its execution.

Hints

- Start by collecting input and ensuring all inputs match properly and are stored in the cats and rivals arrays.
- Consider writing helper functions to calculate a single team's total score given the team indexes as a part of the permutation array.
- Write separate functions to aid your recursive solution.
- Use the permutation function to generate all permutations of cats. For every permutation, score individual teams by calling a helper scorer function on the corresponding indexes of the team, sum all teams' scores, and store the current permutation evaluation. If this current permutation beats the best perm score so far, update it and update the arrangement stored in the tracker.
- Be careful when dealing with floating-point value comparisons. You may want to use EPSILON to account for rounding errors.

Clarifications

For questions or clarifications about the program requirements, please post them in the corresponding discussion thread on Webcourses. If additional clarification is needed, you are welcome to visit the TAs or the instructor during office hours. Requests for help with debugging should be brought to office hours rather than sent by email.

Tentative Rubric (Subject to Change)

A set of test cases will be used to evaluate whether your program produces the expected output. Each failed test case will result in a grade deduction per the rubric below. In addition to the sample test cases, we will use additional (hidden) test cases during grading. Therefore, passing the sample test cases does not guarantee full credit. You are strongly encouraged to thoroughly test your code against a variety of inputs. Because some test cases are hidden, your final score will not be visible until the submission has been fully graded.

Important: Any attempt to hard-code the expected output or otherwise circumvent the grading process constitutes a violation of academic integrity and will be handled in accordance with university policy.

- If your code does not compile on Gradescope, you will receive a score of 0.
- If you modify, omit, or fail to use the required data structures, you may receive a score of 0.
- Failure to use dynamic memory allocation for storing data will result in a score of 0.
- Correct implementation of all required functions and specifications: 40%.
- Properly freeing all dynamically allocated memory with zero memory leaks: 10%.
- Passing all test cases with exact output matching: 50%.